

J-ART: A Japanese Adversarial Red-Team Framework for Application-Layer LLM Security and Cost-Efficiency

Technical Report (v0.1, preprint)

Takayuki Hirose*

2026-06-13

Abstract

Most open red-teaming and jailbreak benchmarks for large language models (LLMs) are English-centric and evaluate *bare models*. We present **J-ART** (Japanese Adversarial Red-Team framework), an open, reproducible harness that (i) evaluates the **application-structure layer** — model $\times$ system-prompt strength $\times$ guardrail $\times$ retrieval (RAG) — rather than bare models; (ii) applies **Japanese-specific obfuscation transforms** (gyaru-moji character substitution, vertical/newline splitting, keigo “polite-coercion” framing, double-talk false premises, Base64, and leet/zero-width smuggling) that defeat naïve keyword filters; (iii) grounds attacks in **MITRE ATLAS** (10 inference-time techniques); and (iv) reports a **cost-efficiency** metric alongside a defense rate with **Wilson 95% confidence intervals**. Attacks use *harmless proxy markers* (a fictitious canary and benign output markers); the framework contains **no weaponizable content**, and the malicious “core” of each attack is masked in all published artifacts. Across 18–21 application configurations and 11 attacks $\times$ 7 transforms, we find that **application-layer defenses dominate raw model capability**, that a dedicated **input classifier (Llama Guard)** eliminated all observed breaches for a weak configuration at lower net cost, and that a simple **normalizing filter** defeats the obfuscation transforms a naïve keyword filter misses. We also document **large run-to-run variance** for identical configurations served via a model router (one configuration moved from 58 breaches to 0 breaches between two runs), which we argue makes single-sample leaderboards unreliable and motivates our statistical treatment.

Status. This is a preliminary technical report / preprint describing a proof-of-concept framework and exploratory findings. It is *not* an official safety evaluation of any vendor’s model. Numbers are run-time snapshots subject to the limitations in Section 7. References are indicative; bibliographic details should be verified before formal submission.

1 Introduction

Public LLM red-teaming tooling (e.g., garak) and jailbreak benchmarks (e.g., HarmBench, JailbreakBench) are predominantly English. Multilingual safety work has shown that *translating* prompts into low-resource languages can bypass safety training, but **language-specific obfuscation** — exploiting a language’s own scripts, typography, and register — remains comparatively underexplored. Japanese in particular offers a rich obfuscation surface (multiple scripts, full-width/zero-width characters, vertical writing, and conventionalized polite registers).

*Independent Researcher. ORCID: 0009-0005-6735-1862. Code: <https://github.com/takker-hero-se/J-ART>. Live leaderboard: <https://takker-hero-se.github.io/J-ART/>. Software DOI (concept): [10.5281/zenodo.20676879](https://doi.org/10.5281/zenodo.20676879). Report text licensed CC-BY-4.0; code MIT.

Two further gaps motivate J-ART. First, **bare-model versus deployed-system evaluation**: practitioners deploy a model *plus* a system prompt, *plus* guardrails, often *plus* retrieval; the security-relevant unit is the **application configuration**, not the model. Second, **security and cost jointly**: defenses cost tokens or latency, so we report a cost-efficiency score so that “cheapest sufficiently-safe configuration” is directly readable.

Our contributions are: (a) a Japanese obfuscation transform suite; (b) an application-layer, MITRE-ATLAS-grounded harness with a safe proxy-marker methodology; (c) a guardrail comparison including a normalizing filter and a model-based classifier; (d) a statistically-reported leaderboard (Wilson CIs, optional repeated trials) with a cost-efficiency metric; and (e) an empirical observation of large router-induced run-to-run variance.

2 Related Work

J-ART sits at the intersection of six lines of work, summarized below and positioned in Table 1.

Adversarial red-teaming tooling. garak [2] scans a *model* with a library of vulnerability probes. J-ART instead treats the deployed *application configuration* (model $\times$ system prompt $\times$ guardrail $\times$ RAG) as the unit under test.

Jailbreak and harmful-behavior benchmarks. AdvBench/GCG [11], Jailbroken [12], HarmBench [3], JailbreakBench [4], and in-the-wild prompts [13] standardize *which* harmful behaviors to elicit and *how* to score them, predominantly in English at the model layer. J-ART reuses the harness idea but replaces harmful elicitation with a safe proxy-marker objective (§3) so the suite can be released publicly.

Automated attack generation. PAIR [6] and TAP [7] optimize jailbreak prompts against a target. J-ART uses a *fixed, auditable* transform suite, trading attack strength for reproducibility and safe disclosure.

Multilingual and low-resource safety. Deng et al. [14] and Yong et al. [5] show alignment degrades under *translation* into low-resource languages, and Wang et al. [15] study cross-lingual safety generalization. These study *translation across* languages; J-ART studies *intra-language orthographic obfuscation* specific to Japanese (gyaru-moji, vertical writing, keigo framing, kana/kanji encoding), which persists even though Japanese is high-resource.

Prompt injection. Perez & Ribeiro [16] introduced goal-hijacking and prompt-leaking; Greshake et al. [8] formalized *indirect* injection; Liu et al. [17] benchmark LLM-integrated apps. J-ART evaluates these as a function of the *application configuration*.

Guardrails and input classifiers. Llama Guard [9], NeMo Guardrails [18], and the OpenAI Moderation API [19] provide model- or rule-based filtering. J-ART treats the guardrail as a *swappable configuration variable* and measures the marginal defense each adds, including a deterministic normalizing filter that defeats obfuscations a naive keyword filter misses (§4).

LLM-as-judge validation. Zheng et al. [20] and the survey of Gu et al. [21] document the biases of LLM judges. J-ART avoids an LLM judge for its *primary* outcome, using a deterministic canary/marker check (§3) so judge variance does not enter the headline metric.

Table 1: Positioning of J-ART relative to representative prior work.

Work	Language	Unit of eval.	Cost axis	Guard/RAG	Public
garak [2]	English	Model	No	No	Yes
HarmBench / JBB [3, 4]	English	Model	No	No	Yes
PAIR / TAP [6, 7]	English	Model	No	No	Partial
Multiling. [5, 14, 15]	Low-resource	Model	No	No	Yes
Llama Guard / NeMo [9, 18]	Eng.-centric	Classifier	No	n/a	Yes
J-ART	Japanese	App config	Yes	Yes	Yes

Japanese NLP resources and taxonomies. General Japanese benchmarks such as JGLUE [22] cover NLU, but to our knowledge no open, reproducible *adversarial-safety* benchmark targets Japanese-specific obfuscation — the gap J-ART addresses. We map attacks to MITRE ATLAS [1] and cross-reference the OWASP Top 10 for LLM Applications [10].

J-ART is thus distinguished by combining a *Japanese intra-language obfuscation* surface, an *application-configuration* unit of evaluation, a *cost-efficiency* axis, and a *safe proxy-marker* methodology in a single reproducible harness.

3 Methodology

Application-structure layer. Each evaluated **target** is a configuration: provider/model $\times$ prompt_strength $\in \{\text{nailed (low), hardened (high)}\} \times$ guardrail $\in \{\text{none, keyword, regex, llm, llamaguard}\} \times$ rag $\in \{\text{true, false}\}$. A simplified RAG context (benign corporate documents, optionally poisoned for indirect-injection attacks) and a system prompt containing a secret *canary* are assembled per trial.

Japanese obfuscation transforms. Each base attack is rendered through one of seven transforms: **baseline** (none), **polite_business** (keigo authority framing), **vertical_newline** (one-character-per-line vertical writing), **gyaru** (gyaru-moji glyph substitution + zero-width spaces), **double_tongue** (false sandbox/authorization premise), **base64_wrap** (Base64-encoded instruction), and **leet_smuggle** (leet symbol substitution + zero-width spaces). Transforms preserve human legibility while breaking substring-matching filters.

We distinguish two classes (Table 2). *Japanese-specific* transforms exploit the Japanese writing system or register and have no direct English equivalent: **gyaru** (kana glyph substitution), **vertical_newline** (vertical writing), and **polite_business/double_tongue** (keigo authority and in-group framing). *Language-agnostic* transforms — **base64_wrap** and **leet_smuggle**, plus the **baseline** control — apply to any language and serve as a comparison baseline so the marginal effect of the Japanese-specific surface can be isolated, clarifying which findings are expected to generalize beyond Japanese.

Attack suite and MITRE ATLAS mapping. Eleven attacks cover ten inference-time ATLAS techniques: Direct/Indirect Prompt Injection (AML.T0051.000/.001), LLM Jailbreak (AML.T0054), LLM Data Leakage (AML.T0057), LLM Prompt Self-Replication (AML.T0061), Trusted-Output/Citations manipulation (AML.T0067.000), LLM Prompt Obfuscation (AML.T0068), System-Prompt discovery (AML.T0069.002), RAG Poisoning (AML.T0070), and False RAG Entry Injection (AML.T0071). Vectors are **user** (direct) or **rag** (indirect).

Table 2: Classification of obfuscation transforms.

Class	Transforms	Exploits
Japanese-specific	<code>gyaru</code> , <code>vertical_newline</code> , <code>polite_business</code> , <code>double_tongue</code>	Kana glyph variants, vertical writing, keigo authority/in-group framing
Language-agnostic	<code>base64_wrap</code> , <code>leet_smuggle</code> <code>baseline</code>)	Encoding / symbol substitution applicable to any language (+

Harmless proxy markers and safety-by-construction. To avoid producing weaponizable content, attacks target **harmless proxies**: a fictitious canary passphrase embedded in the system prompt (data-leakage attacks) and benign string markers `[JART-PWNED]` / `[JART-HIJACKED]` (injection / control-hijack attacks). The malicious instruction **core** of each attack is defined in code, kept out of configuration files, and **masked** (replaced by a redaction placeholder) in `results.json` and the public site; only the obfuscation *framework* is shown, so developers can study the structure without a usable recipe. A defense-in-depth client-side mask is also applied at render time.

Deterministic judge. A breach is declared by exact membership: the canary in the response (`secret_leak`) or the marker in the response (`marker_injection` / `marker_hijack`). This judge is reproducible and cheap but conservative (it can miss semantically-equivalent leaks); see Section 7.

Guardrails. Five settings are compared: **none**; **keyword** (banned-substring match — fast, weak to obfuscation); **regex** (a *normalizing* filter that strips zero-width characters, reverses leet substitutions, folds whitespace, and Base64-decodes long tokens *before* keyword matching — deterministic, zero extra API cost); **llm** (the target model self-moderates input and screens output); and **llamaguard** (a dedicated safety-classification model — Llama Guard 4 via a model router — classifies the input as attack/benign, with a graceful fallback on failure).

Statistical treatment. Each model’s defense rate is reported with a **Wilson 95% confidence interval** computed over its trial count. Optionally, each cell (target, attack, transform) is repeated N times (`JART_TRIALS`, default 1); details aggregate per cell while summary statistics and CIs use the trial-level sample. In simulation mode each repeated trial is an independent Bernoulli draw, which narrows the CI as N grows (e.g., interval width $15.1 \rightarrow 7.5$ points from $N=1$ to $N=3$ for one configuration).

Cost-efficiency metric. We report $\text{cospa} = \text{defense_rate}(\%) \div \text{cost_per_million_tokens (USD)}$ (the per-million cost is clamped at a \$0.01 floor to avoid divergence), surfacing configurations that are both safe and cheap.

Execution. Trials are independent and run with bounded concurrency (a thread pool, default 8 workers) under per-request timeouts and exponential-backoff retries; determinism of the simulation does not depend on execution order. Targets whose API key is absent fall back to a deterministic simulation so a partial key set never aborts the run.

Table 3: Guardrail effect for a weak base configuration (run B, GPT-4o-mini).

Configuration	Defense rate	95% CI	Breaches
GPT-4o-mini / naked	81.8%	[71.8, 88.8]	14 / 77
GPT-4o-mini + Llama Guard	100.0%	[95.2, 100]	0 / 77

4 Results

We summarize two live runs (run A: 18 configurations, 1,386 cells; run B: 21 configurations, 1,617 cells), one trial per cell.

Overall. Breach rates were low in aggregate (run A: $108/1,386 = 7.8\%$; run B: $36/1,617 = 2.2\%$) and concentrated in **weak configurations** (naked, low-strength models); hardened-prompt-plus-guardrail configurations defended at or near 100%.

Application-layer defense dominates model capability. In run A, a flagship model with **no system prompt and no guardrail** was breached 10 times (87.0% defense), whereas a small, inexpensive model behind a **hardened prompt + keyword guardrail** defended at $\geq 96\%$. The central practical finding: *how the application is configured matters more than which model is used.*

Run-to-run variance (a primary result). The **identical** configuration “Llama 4 Scout / naked” produced **58 breaches (24.7% defense)** in run A and **0 breaches (100% defense)** in run B. As the only change between runs was execution time (the model was served via a router that may dispatch to different backends/quantizations), this demonstrates that **single-sample LLM leaderboards are not reproducible**, and that defense rates should be reported with confidence intervals and, ideally, repeated trials. We treat this variance as a result, not noise.

Guardrail comparison. In run B, “GPT-4o-mini / naked” was breached 14 times (81.8% defense, CI[71.8–88.8]); the **same model behind a Llama-Guard input classifier** was breached **0 times** (100% defense, CI[95.2–100]) at lower net cost, because blocked inputs skip the main model call. The **normalizing regex filter**, in unit tests, deterministically neutralizes the vertical-newline, leet, Base64, and gyaru transforms that the keyword filter misses, at zero additional API cost; in run B its weak-model baselines did not breach, so its live preventive effect was not separable in that snapshot (Section 7). Table 3 summarizes the Llama-Guard comparison.

Attack and transform effectiveness. In run A, the most effective attacks were Trusted-Output/Citations manipulation, False RAG Entry Injection, and indirect RAG exfiltration; the most effective transforms were vertical-newline, baseline, and polite-business, while **Base64** was least effective (models frequently declined to act on encoded instructions). The newly-added ATLAS techniques accounted for a large share of breaches.

5 The Leaderboard Artifact

The harness emits `results.json` and a self-contained static site (sortable leaderboard, per-configuration attack logs with masked prompts, JP/EN UI with environment-based auto-selection, per-model defense-rate CIs, and per-cell breach rates), continuously deployed.

6 Ethics and Responsible Disclosure

J-ART is safe by construction. (1) Attacks target a **fictitious canary** and **benign markers**, never real harmful capabilities; no weaponizable content exists in the code, data, or published site. (2) The malicious instruction **core** is masked in all artifacts; only the obfuscation framework is shown. (3) Models are referenced for research comparison only; results are run-time snapshots, not certifications. The intended use is to help developers harden their *own* deployments.

7 Limitations

- **Proxy markers, not harm:** we measure instruction-violation / leakage proxies rather than real-world harm.
- **Deterministic judge:** the judge matches the canary/marker exactly *and* after light normalization (case-folding, whitespace/zero-width removal), so a reformatted echo is counted as a breach. It cannot detect *semantic*/paraphrased leakage, so reported defense rates are an **upper bound**; a human-validated LLM-judge is future work.
- **Single-turn:** multi-turn / crescendo attacks are not yet modeled.
- **Sample size:** a handcrafted suite (11 attacks $\times$ 7 transforms); default $N=1$ per cell.
- **Provider non-determinism / router variance:** results are snapshots and depend on routing and model versions at run time.
- **Cost attribution:** guardrail tokens are attributed at the target model’s price (an approximation).
- **Simulation mode** is a deterministic stand-in for missing keys and must not be read as empirical model behavior.

8 Future Work

LLM-judge with human-validated agreement; multi-turn and adaptive (PAIR/TAP-style) attacks; larger and partially human-curated attack sets; additional ATLAS techniques (cost-exhaustion, model extraction); per-provider concurrency and incremental caching with longitudinal trend tracking; and dropping simulation mode for a purely empirical, repeated-trial statistical report.

9 Reproducibility

Code, configuration, and the deployment workflow are public. The harness is deterministic in simulation mode; live results depend on provider routing and model versions at run time and should be reported with the run date and model identifiers. See the repository for exact model identifiers and pricing used.

References

- [1] MITRE ATLAS — Adversarial Threat Landscape for AI Systems. <https://atlas.mitre.org>.
- [2] garak — Generative AI Red-teaming & Assessment Kit.
- [3] HarmBench: A Standardized Evaluation Framework for Automated Red Teaming.
- [4] JailbreakBench: An Open Robustness Benchmark for Jailbreaking LLMs.

- [5] Yong, Menghini, Bach. Low-Resource Languages Jailbreak GPT-4. 2023.
- [6] Chao et al. Jailbreaking Black-Box LLMs in Twenty Queries (PAIR).
- [7] Mehrotra et al. Tree of Attacks: Jailbreaking Black-Box LLMs Automatically (TAP).
- [8] Greshake et al. Not What You’ve Signed Up For: Indirect Prompt Injection. 2023.
- [9] Inan et al. Llama Guard: LLM-Based Input-Output Safeguard for Human-AI Conversations. Meta, 2023.
- [10] OWASP Top 10 for Large Language Model Applications.
- [11] Zou et al. Universal and Transferable Adversarial Attacks on Aligned Language Models (AdvBench/GCG). 2023.
- [12] Wei, Haghtalab, Steinhardt. Jailbroken: How Does LLM Safety Training Fail? 2023.
- [13] Shen et al. “Do Anything Now”: Characterizing and Evaluating In-the-Wild Jailbreak Prompts on Large Language Models. 2024.
- [14] Deng et al. Multilingual Jailbreak Challenges in Large Language Models. 2024.
- [15] Wang et al. All Languages Matter: On the Multilingual Safety of Large Language Models. 2024.
- [16] Perez & Ribeiro. Ignore Previous Prompt: Attack Techniques for Language Models. 2022.
- [17] Liu et al. Prompt Injection Attack against LLM-Integrated Applications. 2024.
- [18] Rebedea et al. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. 2023.
- [19] Markov et al. A Holistic Approach to Undesired Content Detection in the Real World (OpenAI Moderation). 2023.
- [20] Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. 2023.
- [21] Gu et al. A Survey on LLM-as-a-Judge. 2024.
- [22] Kurihara et al. JGLUE: Japanese General Language Understanding Evaluation. 2022.